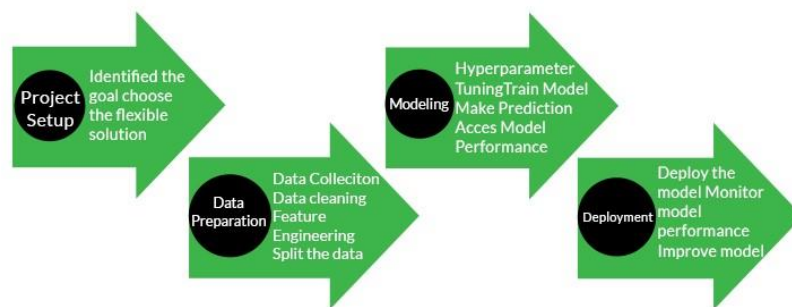


Machine Learning

Machine Learning (ML) is a branch of **Artificial Intelligence (AI)** that works on algorithm developments and statistical models that allow computers to learn from data and make predictions or decisions without being explicitly programmed.

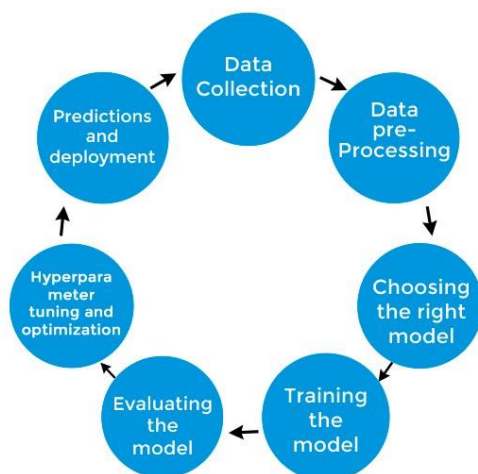
Machine Learning process includes Project Setup, Data Preparation, Modeling and Deployment. The following figure demonstrates the common working process of Machine Learning. It follows some set of steps to do the task; a sequential process of its workflow is as follows –



As the name suggests, it refers to the capability of a machine to learn and exhibit "intelligent behavior" similar to that of humans. Machine learning uses data and algorithms to learn hidden patterns in the data and make predictions on new unseen data.

Stages of Machine Learning

The following are the stages (detailed sequential process) of Machine Learning:



Data Collection – Data collection is an initial step in the process of machine learning. In this stage, it collects data from the different sources such as databases, text files, pictures, sound files, or web scraping. This process organizes the data in an appropriate format, such as a CSV file or database, and makes sure that they are useful for solving your problem.

Data Pre-processing – It is a key step in the process of machine learning, which involves deleting duplicate data, fixing errors, managing missing data either by eliminating or filling it in, and adjusting and formatting the data.

Choosing the Right Model – The next step is to select a machine learning model; once data is prepared, then we apply it to ML models like linear regression, decision trees, and neural networks that may be selected to implement. This selection depends on many factors, such as the kind of data and your problem, the size and type of data, the complexity, and the computational resources.

Training the Model – This step includes training the model from the data so it can make better predictions.

Evaluating the model – When module is trained, the model has to be tested on new data that they haven't been able to see during training.

Hyperparameter Tuning and Optimization – After evaluating the model, you may need to adjust its hyperparameters to make it more efficient. You should try different combinations of parameters and cross-validation to ensure that the model performs well on different data sets.

Predictions and Deployment – When the model has been programmed and optimized, it will be ready to estimate new data. This is done by adding new data to the model and using its output for decision-making or other analysis. The deployment includes its integration into a production environment to make it capable of processing real-world data.

Types of Machine Learning

Machine learning models fall into the following categories –

Supervised Machine Learning – It is a type of machine learning that trains the model using labeled datasets to predict outcomes.

Supervised learning that uses labeled dataset to train algorithms to understand data patterns and predict outcomes.

For example, filtering a mail into inbox or spam folder.

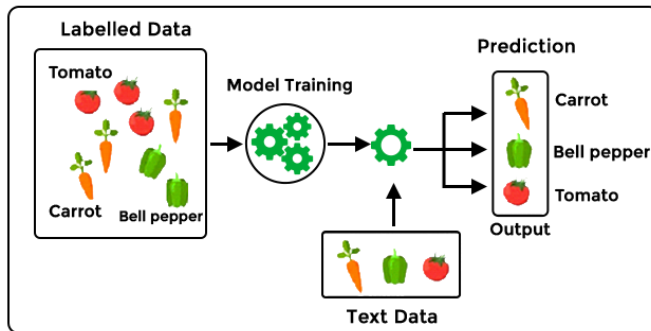
The supervised learning further can be classified into two types – classification and regression.

There are different supervised learning algorithms that are widely used –

- Linear Regression
- Logistic Regression
- Decision Trees
- Random Forest
- K-nearest Neighbor
- Support Vector Machine
- Naive Bayes
- Linear Discriminant Analysis
- Neural Networks

Following are some real-life use cases of supervised learning –

- Image Classification
- Spam Filtering
- House Price Prediction
- Signature Recognition
- Weather Forecasting
- Stock price prediction



2. Unsupervised Machine Learning – It is a type of machine learning that learns patterns and structures within the data without human supervision.

Unsupervised learning is a type of Machine learning that uses unlabeled dataset to discover patterns without any explicit guidance or instruction. For example, customer segmentation i.e, dividing a company's customers into groups that reflect similarity.

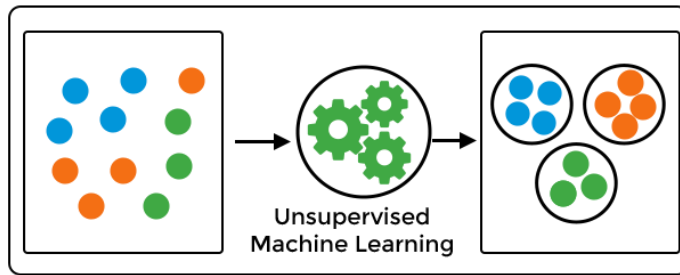
Further, we can classify the unsupervised learning algorithms into three types – clustering, association, and dimensionality reduction.

Followings are some commonly used unsupervised learning algorithms –

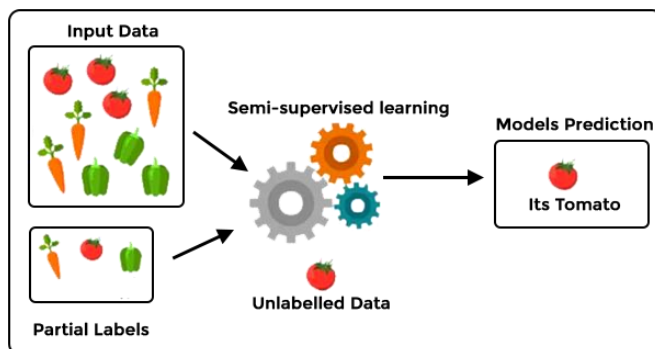
- K-Means Clustering
- Principal Component Analysis(PCA)
- Hierarchical Clustering
- DBSCAN Clustering
- Agglomerative Clustering
- Apriori Algorithm
- Autoencoder
- Restricted Boltzmann machine (RBM)

Some use cases of unsupervised machine learning are as follows –

- Anomaly detection
- Recommendation systems
- Customer segmentation
- Fraud detection
- Natural language processing
- Genetic search



3. Semi-supervised Learning – It is a type of machine learning that is neither fully supervised nor fully unsupervised. The semi-supervised learning algorithms basically fall between supervised and unsupervised learning methods.



4. Reinforcement Machine Learning – It is a type of machine learning model that is similar to supervised learning but does not use sample data to train the algorithm. This model learns by trial and error.

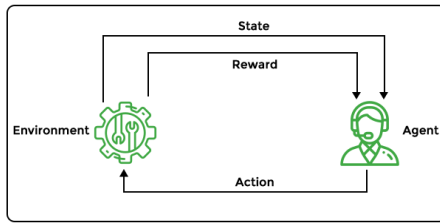
Reinforcement learning algorithms are trained on datasets to make decisions and achieve optimized results by minimizing the trial and error method. For example, Robotics.

Following are some common reinforcement learning algorithms –

- Q-learning
- Markov Decision Process (MDP)
- SARSA
- DQN
- DDPG

Followings are some application examples, where reinforcement learning is used –

- Autonomous vehicles
- Robotics
- Game playing



Common Machine Learning Algorithms

Several machine learning algorithms are commonly used. These include:

- **Neural Networks** – It works like the human brain with many connected nodes. They help to find patterns and are used in language processing, image and speech recognition, and creating images.
- **Linear Regression** – It predicts numbers based on past data. For example, it helps estimate house prices in an area.
- **Logistic Regression** – It predicts like "yes/no" answers and it is useful for spam detection and quality control.
- **Clustering** – It is used to group similar data without instructions and it helps to find patterns that humans might miss.
- **Decision Trees** – They help to classify data and predict numbers using a tree-like structure. They are easy to check and understand.
- **Random forests** – They combine multiple decision trees to improve predictions.

Importance of Machine Learning

Machine Learning is important in automation, extracting insights from data, and decision-making processes. It has its significance due to the following reasons:

- **Data Processing** – Machine learning is useful to analyze large data from social media, sensors, and other sources and help to reveal patterns and insights to improve decision-making.
- **Data-Driven Insights** – Machine learning algorithms find trends and connections in big data that humans might miss, which helps to take better decisions and predictions.
- **Automation** – Machine learning automates the repetitive tasks, reducing errors and saving time.
- **Personalization** – Machine learning is useful to analyze the user preferences to provide personalized recommendations in e-commerce, social media, and streaming services. It helps in many manners, such as to improve user engagement, etc.
- **Predictive Analytics** – Machine learning models use past data to predict future outcomes, which may help for sales forecasts, risk management, and demand planning.
- **Pattern Recognition** – Machine learning is useful in pattern recognition during image processing, speech recognition, and natural language processing.
- **Finance** – Machine learning is used in credit scoring, fraud detection, and algorithmic trading.
- **Retail** – Machine learning helps to enhance the recommendation systems, supply chain management, and customer service.

- **Fraud Detection & Cybersecurity** – Machine learning detects the fraudulent transactions and security threats in real time.
- **Continuous Improvement** – Machine learning models update regularly with new data, which allows them to adapt and improve over time.

Machine learning, as we know, is a subset of artificial intelligence that involves training computer algorithms to automatically learn patterns and relationships in data. Here are some basic concepts of machine learning –

Data

Data is the foundation of machine learning. Without data, there would be nothing for the algorithm to learn from. Data can come in many forms, including structured data (such as spreadsheets and databases) and unstructured data (such as text and images). The quality and quantity of the data used to train the machine learning algorithm are crucial factors that can significantly impact its performance.

Feature

In machine learning, features are the variables or attributes used to describe the input data. The goal is to select the most relevant and informative features that will allow the algorithm to make accurate predictions or decisions. Feature selection is a crucial step in the machine learning process because the performance of the algorithm is heavily dependent on the quality and relevance of the features used.

Model

A machine learning model is a mathematical representation of the relationship between the input data (features) and the output (predictions or decisions). The model is created using a training dataset and then evaluated using a separate validation dataset. The goal is to create a model that can accurately generalize to new, unseen data.

Training

Training is the process of teaching the machine learning algorithm to make accurate predictions or decisions. This is done by providing the algorithm with a large dataset and allowing it to learn from the patterns and relationships in the data. During training, the algorithm adjusts its internal parameters to minimize the difference between its predicted output and the actual output.

Testing

Testing is the process of evaluating the performance of the machine learning algorithm on a separate dataset that it has not seen before. The goal is to determine how well the algorithm generalizes to new, unseen data. If the algorithm performs well on the testing dataset, it is considered to be a successful model.

Overfitting

Overfitting occurs when a machine learning model is too complex and fits the training data too closely. This can lead to poor performance on new, unseen data because the model is too specialized to the training dataset. To prevent overfitting, it is important to use a validation dataset to evaluate the model's performance and to use regularization techniques to simplify the model.

Underfitting

Underfitting occurs when a machine learning model is too simple and cannot capture the patterns and relationships in the data. This can lead to poor performance on both the training and testing datasets. To prevent underfitting, we can use several techniques such as increasing model complexity, collect more data, reduce regularization, and feature engineering..

It is important to note that preventing underfitting is a balancing act between model complexity and the amount of data available. Increasing model complexity can help prevent underfitting, but if there is not enough data to support the increased complexity, overfitting may occur instead. Therefore, it is important to monitor the model's performance and adjust the complexity as necessary.

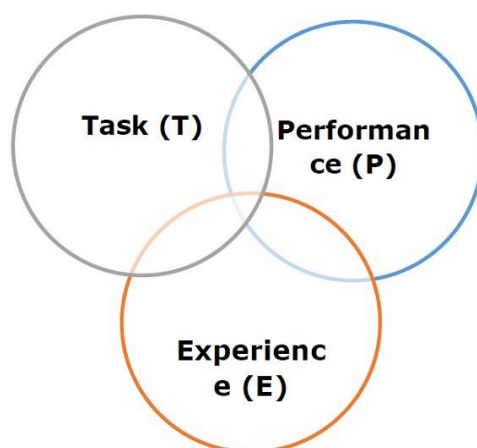
Machine Learning Model

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .

The above definition is basically focusing on three parameters, also the main components of any learning algorithm, namely Task(T), Performance(P) and experience (E). In this context, we can simplify this definition as –

ML is a field of AI consisting of learning algorithms that –

- Improve their performance (P)
- At executing some task (T)
- Over time with experience (E)



Task(T)

From the perspective of problem, we may define the task T as the real-world problem to be solved. The problem can be anything like finding best house price in a specific location or to find best marketing strategy etc. On the other hand, if we talk about machine learning, the definition of task is different because it is difficult to solve ML based tasks by conventional programming approach.

A task T is said to be a ML based task when it is based on the process and the system must follow for operating on data points. The examples of ML based tasks are Classification, Regression, Structured annotation, Clustering, Transcription etc.

Experience (E)

As name suggests, it is the knowledge gained from data points provided to the algorithm or model. Once provided with the dataset, the model will run iteratively and will learn some inherent pattern. The learning thus acquired is called experience(E). Making an analogy with human learning, we can think of this situation as in which a human being is learning or gaining some experience from various attributes like situation, relationships etc. Supervised, unsupervised and reinforcement learning are some ways to learn or gain experience. The experience gained by our ML model or algorithm will be used to solve the task T.

Performance (P)

An ML algorithm is supposed to perform task and gain experience with the passage of time. The measure which tells whether ML algorithm is performing as per expectation or not is its performance (P). P is basically a quantitative metric that tells how a model is performing the task, T, using its experience, E. There are many metrics that help to understand the ML performance, such as accuracy score, F1 score, confusion matrix, precision, recall, sensitivity etc.

Applications of Machine Learning

Machine learning has become the ubiquitous technology that has impacted many aspects of our lives, from business to healthcare to entertainment. Machine learning helps make decisions and find all possible solutions to a problem which improves the efficiency of work in every sector.

Some of the successful machine learning applications are chatbots, language translation, face recognition, recommendation systems, autonomous vehicles, object detection, medical image analysis, etc. Here are some popular applications of machine learning –

- Image and Speech Recognition
- Natural Language Processing
- Finance Sector
- E-commerce and Retail
- Automotive Sector
- Computer Vision
- Manufacturing and Industries
- Healthcare Sector

Image and Speech Recognition

Image and speech recognition are two areas where machine learning has significantly improved. Machine learning algorithms are used in applications such as facial recognition, object detection, and speech recognition to accurately identify and classify images and speech.

Natural Language Processing

Natural Language Processing (NLP) is a field of computer science that deals with the interaction between computers and humans using natural language. NLP uses machine learning algorithms to identify parts of speech, sentiment and other aspects of text. It analyzes, understands, and generates human language. It is currently all over the internet which includes translation software, search engines, chatbots, grammar correction software and voice assistants, etc.

Here is a list of some applications of machine learning in natural language processing –

- Sentiment Analysis
- Speech synthesis
- Speech recognition
- Text classification
- Chatbots
- Language translation
- Caption generation
- Document summarization
- Question answering
- Autocomplete in search engines

Finance Sector

The role of machine learning in finance is to maintain secure transactions. Also, in trading, the data is converted to information for the decision-making process. Some applications of machine learning in the finance sector are –

1. Fraud Detection

Machine learning is widely used in the finance industry for fraud detection. Fraud detection is a process of using a machine learning model to monitor transactions and understand patterns in the dataset to identify fraudulent and suspicious activities.

Machine learning algorithms can analyze vast amounts of transactional data to detect patterns and anomalies that may indicate fraudulent activity, helping to prevent financial losses and protect customers.

2. Algorithmic Trading

Machine learning algorithms are used to identify complex patterns in the large dataset to discover trading signals which might not be possible for humans.

Some other applications of machine learning in the finance sector are as follows –

- Stock market analysis and forecasting
- Credit risk assessment and management
- Security analysis and portfolio optimization
- Asset evaluation and management

E-commerce and Retail

Machine learning is used to enhance the business in e-commerce and retail sector through recommendation systems and target advertising which improve user experience. Machine learning makes the process of marketing easy by performing repetitive tasks. Some tasks where Machine learning is applied are:

1. Recommendation Systems

Recommendation systems are used to provide personalized recommendations to users based on their past behavior and preferences and previous interaction with the website. Machine learning algorithms are used to analyze user data and generate recommendations for products, services, and content.

2. Demand Forecasting

Companies use machine learning to understand the future demand for their product or services based on various factors like market trends, customer behavior and historical data regarding sales.

3. Customer Segmentation

Machine learning can be used to segment customers into particular groups with similar characteristics. The purpose of customer segmentation is to understand customer behavior and target them with personalized experience.

Manufacturing and Industries

Machine learning is also used in manufacturing and industries to keep a check on the working conditions of machinery. Predictive Maintenance is used to identify defects in operational machines and equipment to avoid unexpected outages. This detection of anomalies would also help with regular maintenance.

Predictive maintenance is a process of using machine learning algorithms to predict when maintenance will be required on a machine, such as a piece of equipment in a factory. By analyzing data from sensors and other sources, machine learning algorithms can detect patterns that indicate when a machine is likely to fail, enabling maintenance to be performed before the machine breaks down.

Healthcare Sector

Machine learning has also found many applications in the healthcare industry. For example, machine learning algorithms can be used to analyze medical images and detect diseases such as cancer or to predict patient outcomes based on their medical history and other factors.

Some applications of machine learning in healthcare are discussed below –

1. Medical Imaging and Diagnostics

Machine learning in medical imaging is used to analyze the patterns in the image that indicate the presence of a particular disease.

2. Drug Discovery

Machine learning techniques are used to analyze vast datasets, to predict the biological activity of compounds, and to identify potential drugs for a disease by analyzing its chemical structures.

3. Disease Diagnosis

Machine learning may also be used to identify some types of diseases. Breast cancer, heart failure, Alzheimer's disease, and pneumonia are some examples of such diseases that can be identified using machine learning algorithms.

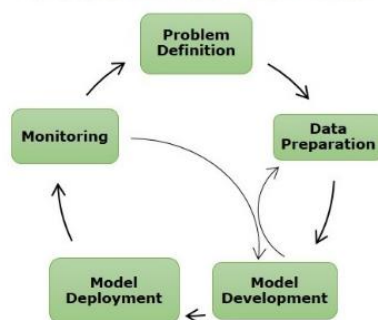
Machine Learning Life Cycle

The machine learning life cycle is an iterative process that moves from a business problem to a machine learning solution. It is used as a guide for developing a machine learning project to solve a problem. It provides us with instructions and best practices to be used in each phase while developing ML solutions.

The machine learning life cycle is a process that involves several phases from problem identification to model deployment and monitoring. While developing an ML project, each step in the life cycle is revisited many times through these phases. The stages/ phases involved in the end to end machine life cycle process are as follows –

- Problem Definition
- Data Preparation
- Model Development
- Model Deployment
- Monitoring and Maintenance

MACHINE LEARNING LIFE CYCLE



Problem Definition

The first step in the machine learning life cycle is to identify the problem you want to solve. It is a crucial step which helps you start building a machine learning solution for a problem. This process of identifying a problem would establish an understanding about what the output might be, scope of the task and its objective.

As this step lays the foundation for building a machine learning model, the problem definition has to be clear and concise.

This stage involves understanding the business problem, defining the problem statement, and identifying the success criteria for the machine learning model.

Data Preparation

Data preparation is a process to prepare data for analysis by performing data exploration, feature engineering, and feature selection. Data exploration involves visualizing and understanding the data, while feature engineering involves creating new features from the existing data. Feature selection involves selecting the most relevant features that will be used to train the machine learning model.

Data preparation process includes collecting data, preprocessing data, and feature engineering & feature selection. This stage generally also includes exploratory data analysis.

Let's discuss each step involved in the data preparation phase of machine learning life cycle process –

1. Data Collection

After the problem statement is analyzed, the next step would be collecting data. This involves gathering data from various sources which is given as a raw material to the machine learning model. Few features that are considered while collecting data are –

- **Relevant and usefulness** – The data collected has to be relevant to the problem statement, and also should be useful enough to train the machine learning model efficiently.
- **Quality and Quantity** – The quality and quantity of the data collected would directly impact the performance of the machine learning model.
- **Variety** – Make sure that the data collected is diverse so that the model can be trained with multiple scenarios to recognize the patterns.

There are various sources from where the data can be collected like surveys, existing databases, and online platforms like Kaggle. The sources may be primary data which includes data collected exclusively for the problem statement while the secondary data includes the existing data.

2. Data Preprocessing

The data collected often might be unstructured and messy which causes it to negatively affect the outcomes, hence pre processing data is important to improve the accuracy and performance of the machine learning model. Issues that have to be addressed are **missing values, duplicate data, invalid data and noise**.

This step of data preprocessing also called data wrangling is intended to make the data more consumable and useful for analytics.

3. Analyzing Data

After the data is all sorted, it is time to understand the data that is collected. The data is visualized and statistically summarized to gain insights.

Various tools like Power BI, Tableau are used to visualize data which helps in understanding the patterns and trends in the data. This analysis will help to make choices in feature engineering and model selection.

4. Feature Engineering and Selection

A 'Feature' is an individual measurable quantity which is preferably observed when the machine learning model is being trained. **Feature Engineering** is the process of creating new features or enhancing the existing ones to accurately understand the patterns and trends in the data.

Feature selection involves the process of picking up features that are consistent and more relevant to the problem statement. The process of feature engineering and selection are used to reduce the size of the dataset which is important to tackle the issue of growing data.

Model Development

In the model development phase, the machine learning model is built using the prepared data. The model building process involves selecting the appropriate machine learning algorithm, algorithm training, tuning the hyperparameters of the algorithm, and evaluating the performance of the model using cross-validation techniques.

This phase mainly consists of three steps, model selection, model training, and model evaluation. Let's discuss these three steps in detail –

1. Model Selection

Model selection is a crucial step in the machine learning workflow. The decision of choosing a model depends on basic features like characteristics of the data, complexity of the problem, desired outcomes and how well it aligns with the defined problem. This step affects the outcomes and performance metrics of the model.

2. Model Training

In this process, the algorithm is fed with a preprocessed dataset to identify and understand the patterns and relationships in the specified features.

Consistent training of a model by adjusting parameters would improve the prediction rate and enhance accuracy. This step makes the model reliable in real-world scenarios.

3. Model Evaluation

In model evaluation, the performance of the machine learning model is evaluated using a set of evaluation metrics. These metrics measure the accuracy, precision, recall, and F1 score of the model. If the model has not achieved desired performance, the model is tuned to adjust hyper parameters and improve the predictive accuracy. This continuous iteration is essential to make the model more accurate and reliable.

If the model's performance is still not satisfactory, it may be necessary to return to the model selection stage and continue to model training and evaluation to improve the model's performance.

Model Deployment

In the model deployment phase, we deploy the machine learning model into production. This process involves integrating the tested model with existing systems to make it available to users, management or other purposes. This also involves testing the model in a real-world scenario.

Two important factors that have to be checked before deploying are whether the model is portable i.e, the ability to transfer the software from one machine to another and scalable i.e, the model need not be redesigned to maintain performance.

Monitor and Maintenance

Monitoring in machine learning involves techniques to measure the model performance metrics and to detect issues in the models. After an issue is detected, the model has to be trained with new data or the architecture has to be modified.

Sometimes when the issue detected in the designed model cannot be solved with training it with new data, the issue becomes the problem statement. So, the machine learning life cycle revamps from analyzing the problem again to develop an improved model.

The machine learning life cycle is an iterative process, and it may be necessary to revisit previous stages to improve the model's performance or address new requirements. By following the machine learning life cycle, data scientists can ensure that their machine learning models are effective, accurate, and meet the business requirements.

Data preprocessing

- Data preprocessing is a crucial step in the machine learning lifecycle. Real-world data is often incomplete, inconsistent, noisy, and stored in formats that are not directly suitable for machine learning algorithms. Therefore, before building predictive models, the data must be cleaned, transformed, and prepared to ensure accuracy and reliability.

Data Cleaning involves identifying and correcting errors, removing duplicate records, handling inconsistencies, and dealing with noisy data to improve data quality. Clean data leads to better model performance and more reliable insights.

Feature Engineering is the process of creating, selecting, and transforming variables (features) from raw data to improve the predictive power of machine learning models. Effective feature engineering helps algorithms discover meaningful patterns and relationships within the data.

Handling Missing Values is essential because missing data can negatively impact model accuracy. Common techniques include deleting incomplete records, replacing missing values with statistical measures such as mean, median, or mode, and using advanced imputation methods.

Normalization is a scaling technique used to bring numerical features into a common range. Since many machine learning algorithms are sensitive to the scale of input features, normalization helps improve convergence speed and model performance.

Encoding is the process of converting categorical data into numerical form so that machine learning algorithms can process it. Popular encoding techniques include Label Encoding, One-Hot Encoding, and Ordinal Encoding.

Together, these preprocessing techniques transform raw data into a structured and machine-readable format, forming the foundation for effective machine learning model development and accurate predictive analytics.

Data Cleaning

Data cleaning is the process of preparing raw data by detecting and correcting errors so it can be effectively used for analysis. It is a foundational step in data preprocessing that ensures datasets are suitable for analytical, statistical and machine learning tasks.

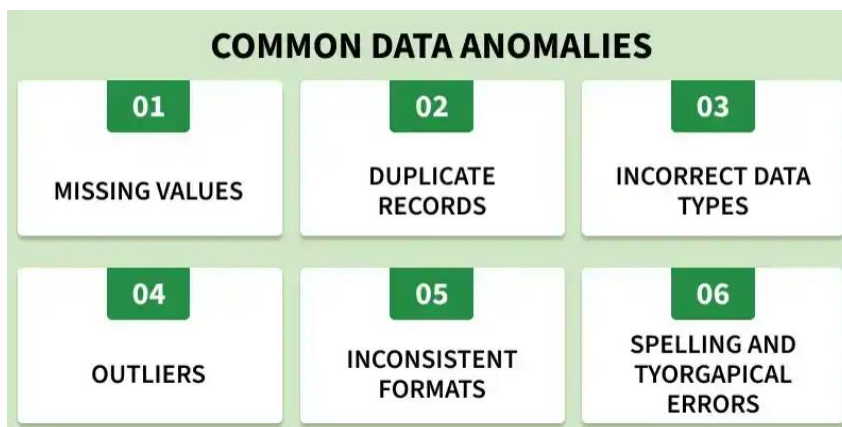
- Raw data is often noisy, incomplete and inconsistent which can negatively impact the accuracy of the model.
- Clean datasets are also important in EDA (Exploratory Data Analysis), which enhances the interpretability of data so that the right actions can be taken based on insights.



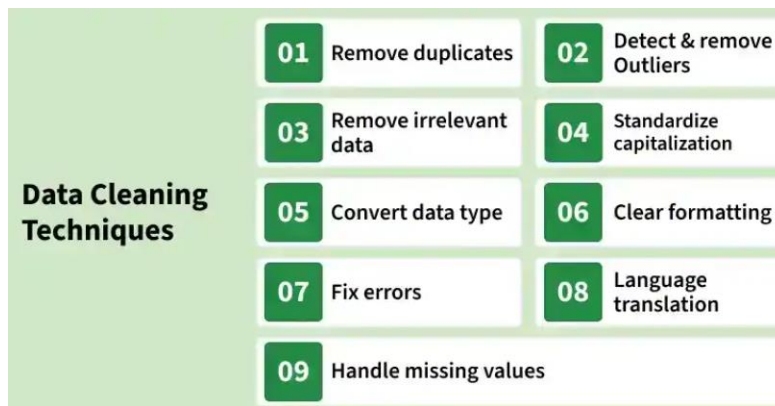
Common Data Anomalies

Data quality issues can arise from human errors, system failures or problems during data collection and integration. Some of the most common data quality challenges include:

- **Missing values:** Incomplete records can reduce statistical power and introduce bias into analysis.
- **Duplicate records:** Repeated entries may overrepresent certain observations resulting in skewed outcomes.
- **Incorrect data types:** Mismatched formats, such as text stored in numeric fields can cause calculation errors and analysis failures.
- **Outliers and anomalies:** Extremely high or low values can distort statistical measures and influence model performance.
- **Inconsistent formats:** Variations in date formats, text casing or measurement units can create issues when merging or comparing datasets.
- **Spelling and typographical errors:** Errors in text fields can lead to incorrect grouping, classification or interpretation of categorical data.

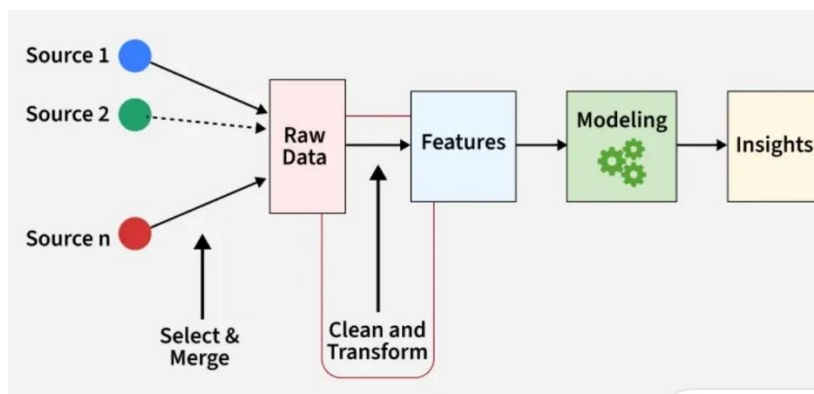


Data Cleaning Process



Feature Engineering

Feature Engineering is the process of selecting, creating or modifying features like input variables or data to help machine learning models learn patterns more effectively. It involves transforming raw data into meaningful inputs that improve model accuracy and performance.



Feature Engineering Architecture

This step may include handling missing values, encoding categories, scaling numbers, creating new features or combining existing ones. It helps turn messy real-world data into a form that models can understand and use for better predictions.

Importance of Feature Engineering

- **Improve accuracy:** Choosing the right features helps the model learn better, leading to more accurate predictions.
- **Reduce overfitting:** Using fewer, more important features helps the model avoid memorizing the data and perform better on new data.
- **Boost interpretability:** Well-chosen features make it easier to understand how the model makes its predictions.
- **Enhance efficiency:** Focusing on key features speeds up the model's training and prediction process, saving time and resources.

Processes involved in Feature Engineering

1. Feature Creation: Feature creation involves generating new features from domain knowledge or by observing patterns in the data. It can be:

- **Domain-specific:** Created based on industry knowledge like business rules.
- **Data-driven:** Derived by recognizing patterns in data.
- **Synthetic:** Formed by combining existing features.

2. Feature Transformation: Transformation adjusts features to improve model learning:

- **Normalization & Scaling:** Adjust the range of features for consistency.
- **Encoding:** Converts categorical data to numerical form i.e one-hot encoding.
- **Mathematical transformations:** Like logarithmic transformations for skewed data.

3. Feature Extraction: Transform existing features into a lower-dimensional or more informative representation (e.g., PCA).

- **Dimensionality reduction:** Techniques like PCA reduce features while preserving important information.
- **Aggregation & Combination:** Summing or averaging features to simplify the model.

4. Feature Selection: Feature selection involves choosing a subset of relevant features to use:

- **Filter methods:** Based on statistical measures like correlation.
- **Wrapper methods:** Select based on model performance.
- **Embedded methods:** Feature selection integrated within model training.

5. Feature Scaling: Scaling ensures that all features contribute equally to the model:

- **Min-Max scaling:** Rescales values to a fixed range like 0 to 1.
- **Standard scaling:** Standardizes features to have mean 0 and variance 1

Steps in Feature Engineering

Feature engineering can vary depending on the specific problem but the general steps are:

1. **Data Cleaning:** Identify and correct errors or inconsistencies in the dataset to ensure data quality and reliability.
2. **Data Transformation:** Transform raw data into a format suitable for modeling including scaling, normalization and encoding.
3. **Feature Extraction:** Create new features by combining or deriving information from existing ones to provide more meaningful input to the model.
4. **Feature Selection:** Choose the most relevant features for the model using techniques like correlation analysis, mutual information and stepwise regression.
5. **Feature Iteration:** Continuously refine features based on model performance by adding, removing or modifying features for improvement.

Handling Missing Values

Missing values appear when some entries in a dataset are left blank, marked as NaN, None or special strings like "Unknown". If not handled properly, they can reduce accuracy, create bias and break algorithms that require complete data.

Keeps models accurate

Prevents bias

Maintains sample size

Allows correct use of ML algorithms

Reasons Behind Missing Values in the Dataset

Data can be missing from a dataset for several reasons and understanding the cause is important for selecting the most effective way to handle it. Common reasons for missing data include:

- **Technical issues:** Failed data collection or errors during data transmission.
- **Human errors:** Mistakes like incorrect data entry or oversights during data processing.
- **Privacy concerns:** Missing sensitive or personal information due to confidentiality policies.
- **Data processing issues:** Errors that occur during data preparation.

Types of Missing Values

- **MCAR (Missing Completely at Random):** Missingness occurs randomly and is not related to any variable in the dataset.
- **MAR (Missing at Random):** Missingness depends on other observed variables, not on the missing value itself.
- **MNAR (Missing Not at Random):** Missingness is directly related to the value that is missing (e.g., high-income individuals not reporting income).

Methods for Identifying Missing Data

Detecting and managing missing data is important for data analysis.

some useful functions for detecting, removing and replacing null values in Pandas DataFrame.

Functions	Descriptions
<code>.isnull()</code>	Detect missing values
<code>.notnull()</code>	Detect non-missing values
<code>.info()</code>	Summary with missing counts
<code>.isna()</code>	Same as <code>isnull()</code> .
<code>dropna()</code>	Remove missing rows/columns
<code>fillna()</code>	Fill missing values
<code>replace()</code>	Replace specific values
<code>drop_duplicates()</code>	Remove duplicate rows
<code>unique()</code>	Get unique values in a Series/DataFrame

Data Normalization

Data normalization is a preprocessing method that resizes the range of feature values to a specific scale, usually between 0 and 1. It is a feature scaling technique used to transform data into a standard range. Normalization ensures that features with different scales or units contribute equally to the model and improves the performance of many machine learning algorithms.

Key Features of Normalization:

- Maps the minimum and maximum of a feature to a defined range
- Preserves the relative relationships of the original data
- Useful for algorithms that rely on distance metrics such as k-Nearest Neighbours and clustering

Machine learning models often assume that all features contribute equally. Features with different scales can dominate the model's behavior if not scaled properly. Using normalization, we can:

- **Ensure Equal Contribution of Features:** Prevents features with larger scales from dominating models that are sensitive to magnitude such as K-Nearest Neighbours or neural networks.
- **Improve Model Performance:** Algorithms that rely on distances or similarities (KNN, K-Means clustering) perform better when features are normalized.
- **Accelerate Convergence:** Helps gradient-based algorithms like logistic regression or neural networks converge faster by keeping feature values in a similar range.
- **Maintain Interpretability of Scales:** By converting all features to a common range, it's easier to understand their relative impact on predictions.

Difference Between Normalization and Standardization

Standardization, also called Z-score normalization is a separate technique. It transforms data so that it has a mean of 0 and a standard deviation of 1.

Key Features of Standardization:

- Centers the data around zero
- Scales according to the variability (standard deviation)
- Useful for algorithms that assume Gaussian distributions such as linear regression, logistic regression and neural networks.
- Less sensitive to outliers compared to Min-Max normalization.

Feature	Normalization (Min-Max)	Standardization (Z-score)
Goal	Rescale data to a specific range	Center data to mean 0, SD 1
Range of values	Fixed (e.g., 0–1)	Not fixed
Effect of outliers	Sensitive	Less sensitive
Assumes data distribution	No	Assumes roughly Gaussian
Use case	Distance-based algorithms	Algorithms assuming Gaussian or regularization
Example	Scaling pixel values to [0,1]	Scaling test scores to z-scores

Different Data Normalization Techniques

There are several techniques to normalize data, each transforming values to a common scale in different ways

1. Min-Max Normalization

Min-Max normalization rescales a feature to a specific range, typically [0, 1]:

$$X_{\text{normalized}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

- The minimum value maps to 0
- The maximum value maps to 1
- Other values are scaled proportionally

2. Decimal Scaling

Decimal scaling normalizes data by shifting the decimal point of values:

$$v' = \frac{v}{10^j}$$

- j is the smallest integer such that the maximum absolute value of v' is less than 1

3. Logarithmic Transformation

Log transformation compresses large values and spreads out small values:

$$X' = \log(X + 1)$$

- Reduces skewness in data
- Stabilizes variance across features

4. Unit Vector (Vector) Normalization

Scales a data vector to have a magnitude of 1:

$$X' = \frac{X}{||X||}$$

- Commonly used in text mining and machine learning algorithms like KNN
- Preserves direction but normalizes magnitude

Encoding

Categorical data refers to variables that belong to distinct categories such as labels, names or types. Since most machine learning algorithms require numerical inputs, encoding categorical data to numerical data becomes important. Proper encoding ensures that models can interpret categorical variables effectively, leading to improved predictive accuracy and reduced bias.

Types of Categorical Data

1. Nominal Data: Nominal data consists of categories without any inherent order or ranking. These are simple labels used to classify data.

- **Example:** 'Red', 'Blue', 'Green' (Car Color).
- **Encoding Options:** One-Hot Encoding or Label Encoding, depending on the model's needs.

2. Ordinal Data: Ordinal data includes categories with a defined order or ranking, where the relationship between values is important.

- **Example:** 'Low', 'Medium', 'High' (Car Engine Power).
- **Encoding Options:** Ordinal Encoding.

Techniques to perform Categorical Data Encoding

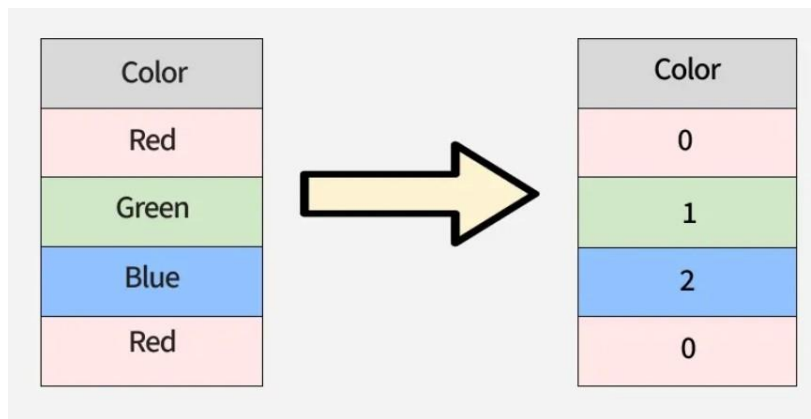
Categorical Data Encoding Techniques

- One-Hot Encoding** : Converts categories into binary columns.
- Label Encoding** : Assigns a unique integer to each category.
- Ordinal Encoding** : Maps categories to ordered integers.
- Binary Encoding** : Converts categories into binary digits.
- Count (Frequency) Encoding** : Replaces categories with their frequency or count.
- Target Encoding** : Replaces categories with the mean target value for each category.

1. Label Encoding

Label Encoding assigns each category a unique integer. It is simple and memory-efficient but may unintentionally imply an order among categories when none exists.

- Used in tree-based models like Decision Trees or XGBoost.
- Pros**: Simple and memory-efficient.
- Cons**: Introduces implicit order which may be misinterpreted by non-tree models when used with nominal data.



Label Encoding

```
from sklearn.preprocessing import LabelEncoder  
  
data = ['Red', 'Green', 'Blue', 'Red']  
  
le = LabelEncoder()  
encoded_data = le.fit_transform(data)  
print(f"Encoded Data: {encoded_data}")
```

Output:

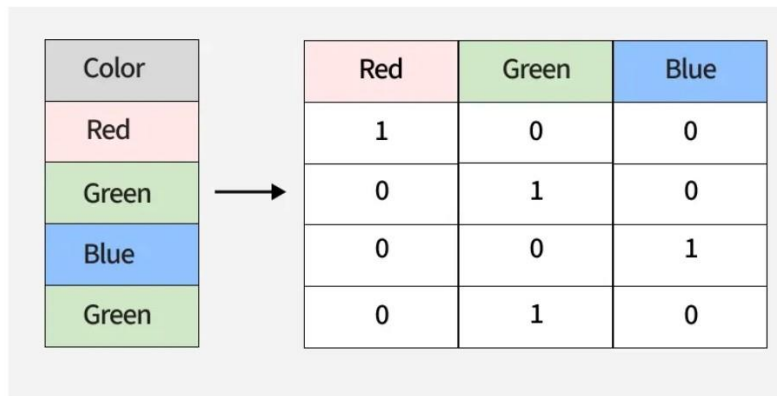
Encoded Data: [0 1 2 0]

2. One-Hot Encoding

One-Hot Encoding converts categories into binary columns with each column representing one category. It prevents false ordering but can lead to high dimensionality if there are many unique values.

- Used in linear models, logistic regression and neural networks.

- **Pros:** Does not assume order; widely supported.
- **Cons:** Can cause high dimensionality and sparse data when feature has many categories.



```
import pandas as pd

data = ['Red', 'Blue', 'Green', 'Red']

df = pd.DataFrame(data, columns=['Color'])
one_hot_encoded = pd.get_dummies(df['Color'])

print(one_hot_encoded)
```

Output:

	Blue	Green	Red
0	False	False	True
1	True	False	False
2	False	True	False
3	False	False	True

output

3. Ordinal Encoding

Ordinal Encoding maps categories to integers while preserving their natural order. This works well for ordered data like ratings but is not suitable for nominal variables.

- Used for ordered features like ratings or education levels.
- **Pros:** Maintains order; reduces dimensionality.
- **Cons:** Not suitable for nominal categories.

Original Encoding	Ordinal Encoding
Poor	1
Good	2
Very Good	3
Excellent	4

Ordinal Encoding

```
from sklearn.preprocessing import OrdinalEncoder
data = [['Low'], ['Medium'], ['High'], ['Medium'], ['Low']]

encoder = OrdinalEncoder(categories=[['Low', 'Medium', 'High']])
encoded_data = encoder.fit_transform(data)

print(f"Encoded Ordinal Data: {encoded_data}")
```

Output:

```
Encoded Ordinal Data: [[0.]
 [1.]
 [2.]
 [1.]
 [0.]]
```

output

4. Target Encoding

Target Encoding also known as Mean Encoding is a technique where each category in a feature is replaced by the mean of the target variable for that category.

- Useful for high-cardinality features like ZIP codes or product IDs.
- **Pros:** Captures relationship to target variable.
- **Cons:** Risk of overfitting, also must apply smoothing/statistical techniques.

Category	Mean Target (Experience)
Engineer	0.8
Lawyer	0.5
Nurse	0.3

Target Encoding

```
import pandas as pd
import category_encoders as ce

df = pd.DataFrame(
    {'City': ['London', 'Paris', 'London', 'Berlin'], 'Target': [1, 0, 1, 0]}
)

encoder = ce.TargetEncoder(cols=['City'])
df_tgt = encoder.fit_transform(df['City'], df['Target'])

print(f"Encoded Target Data:\n{df_tgt}")
```

Output:

```
Encoded Target Data:
      City
0  0.570926
1  0.434946
2  0.570926
3  0.434946
```

output

5. Binary Encoding

Binary encoding represents categories as binary codes and splits them across multiple columns. It is efficient for high-cardinality data but slightly more complex to implement.

- Applied in high-cardinality text/NLP tasks to save memory.
- **Pros:** Reduces dimensionality, more memory-efficient than one-hot encoding.
- **Cons:** Slightly more complex; requires careful handling of missing values.

Candidate Solution (A)

Activation function	Learning rate	Momentum	1* layer	2* layer	
1 0	0 1 1 0 1 0	1 0 1 0	1 0 1 0 0	1 1 0 1 1	← binary value
2	0.40625	0.625	20	27	← decimal value

Candidate Solution (B)

Activation function	Learning rate	Momentum	1* layer	2* layer	
1 1	0 0 0 0 1 0	0 0 1 0	0 0 1 0 0	0 0 0 0 0	← binary value
3	0.03125	0.125	4	0	← decimal value

Binary Encoding

```
import category_encoders as ce
data = ['Red', 'Green', 'Blue', 'Red']
encoder = ce.BinaryEncoder(cols=['Color'])
encoded_data = encoder.fit_transform(pd.DataFrame(data, columns=['Color']))
print(encoded_data)
```

Output:

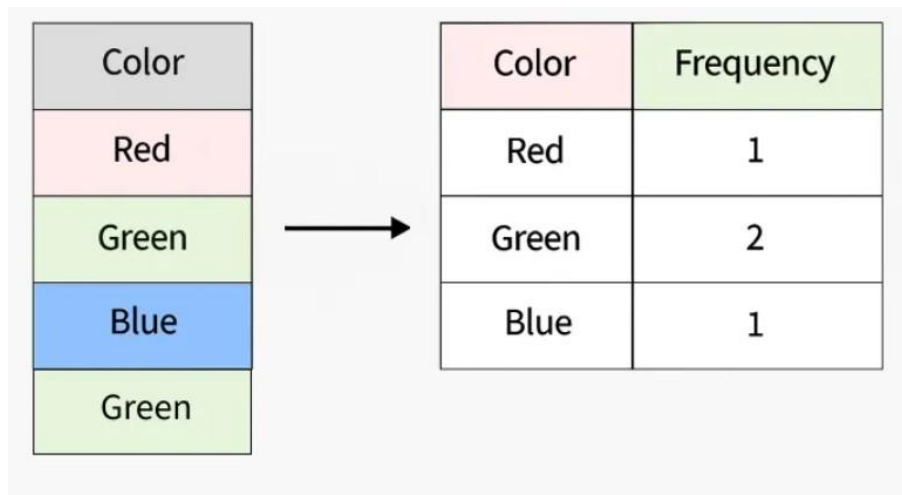
	Color_0	Color_1
0	0	1
1	1	0
2	1	1
3	0	1

output

6. Frequency Encoding

Frequency Encoding assigns categories values based on how often they occur in the dataset. It is simple and compact but can introduce data leakage if applied improperly.

- Effective in retail, e-commerce or clickstream data for popularity trends.
- **Pros:** Low computational and storage requirements.
- **Cons:** Can introduce data leakage if not handled properly.



Frequency Encoding

```
import pandas as pd
data = ['Red', 'Green', 'Blue', 'Red', 'Red']
series_data = pd.Series(data)
frequency_encoding = series_data.value_counts()

encoded_data = [frequency_encoding[x] for x in data]
print("Encoded Data:", encoded_data)
```

Output:

Encoded Data: [np.int64(3), np.int64(1), np.int64(1), np.int64(3), np.int64(3)]

Evaluation Metrics in Machine Learning

Evaluation metrics are used to measure how well a machine learning model performs. They help assess whether the model is making accurate predictions and meeting the desired goals. This is important because:

- **Model Performance** : Measures how well the model works
- **Different Tasks** : Used for classification, regression and clustering
- **Right Metric Choice** : Helps select the best way to evaluate a model
- **Better Decisions** : Ensures the model meets its objectives

Classification Metrics

Classification problems aim to predict discrete categories. To evaluate the performance of classification models, we use the following metrics:

1. Accuracy

Accuracy is a fundamental metric used for evaluating the performance of a classification model. It tells us the proportion of correct predictions made by the model out of all predictions.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

2. Precision

It measures how many of the positive predictions made by the model are actually correct. It's useful when the cost of false positives is high such as in medical diagnoses where predicting a disease when it's not present can have serious consequences.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Where:

- TP = True Positives
- FP = False Positives

3. Recall

Recall or Sensitivity measures how many of the actual positive cases were correctly identified by the model. It is important when missing a positive case (false negative) is more costly than false positives.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Where:

- FN = False Negatives

4. F1 Score

The F1 Score is the harmonic mean of precision and recall. It is useful when we need a balance between precision and recall as it combines both into a single number. A high F1 score means the model performs well on both metrics. Its range is [0,1].

Lower recall and higher precision gives us great accuracy but then it misses a large number of instances. More the F1 score better will be performance. It can be expressed mathematically in this way:

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

5. Logarithmic Loss (Log Loss)

Log loss measures the uncertainty of the model's predictions. It is calculated by penalizing the model for assigning low probabilities to the correct classes. This metric is used in multi-class classification and is helpful when we want to assess a model's confidence in its predictions. If there are N samples belonging to the M class, then we calculate the Log loss in this way:

$$\text{Logarithmic Loss} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^M y_{ij} \cdot \log(p_{ij})$$

Where:

- y_{ij} = Actual class (0 or 1) for sample i and class j

- p_{ij} = Predicted probability for sample i and class j

The goal is to minimize Log Loss, as a lower Log Loss shows higher prediction accuracy.

6. Area Under Curve (AUC) and ROC Curve

It is useful for binary classification tasks. The AUC value represents the probability that the model will rank a randomly chosen positive example higher than a randomly chosen negative example. AUC ranges from 0 to 1 with higher values showing better model performance.

1. True Positive Rate(TPR)

Also known as sensitivity or recall, the True Positive Rate measures how many actual positive instances were correctly identified by the model. It answers the question: "Out of all the actual positive cases, how many did the model correctly identify?"

Formula:

$$TPR = \frac{TP}{TP+FN}$$

Where:

- TP = True Positives (correctly predicted positive cases)
- FN = False Negatives (actual positive cases incorrectly predicted as negative)

2. True Negative Rate(TNR)

Also called specificity, the True Negative Rate measures how many actual negative instances were correctly identified by the model. It answers the question: "Out of all the actual negative cases, how many did the model correctly identify as negative?"

Formula:

$$TNR = \frac{TN}{TN + FP}$$

Where:

- TN = True Negatives (correctly predicted negative cases)
- FP = False Positives (actual negative cases incorrectly predicted as positive)

3. False Positive Rate(FPR)

It measures how many actual negative instances were incorrectly classified as positive. It's a key metric when the cost of false positives is high such as in fraud detection.

Formula:

$$FPR = \frac{FP}{FP + TN}$$

Where:

- FP = False Positives (incorrectly predicted positive cases)
- TN = True Negatives (correctly predicted negative cases)

4. False Negative Rate(FNR)

It measures how many actual positive instances were incorrectly classified as negative. It answers: "Out of all the actual positive cases, how many were misclassified as negative?"

Formula:

$$FNR = \frac{FN}{FN + TP}$$

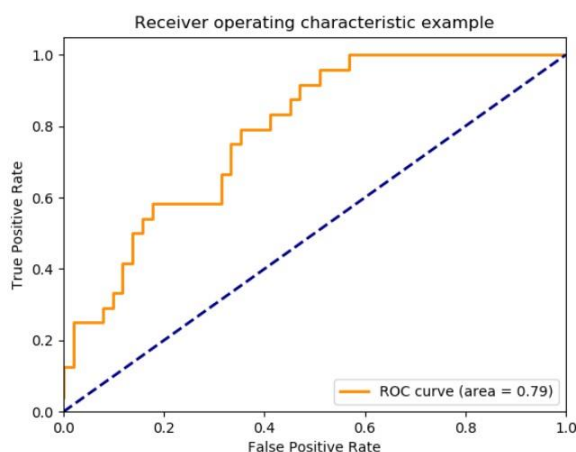
Where:

- FN = False Negatives (incorrectly predicted negative cases)
- TP = True Positives (correctly predicted positive cases)

ROC Curve

It is a graphical representation of the True Positive Rate (TPR) vs the False Positive Rate (FPR) at different classification thresholds. The curve helps us visualize the trade-offs between sensitivity (TPR) and specificity (1 - FPR) across various thresholds. Area Under Curve (AUC) quantifies the overall ability of the model to distinguish between positive and negative classes.

- AUC = 1: Perfect model (always correctly classifies positives and negatives).
- AUC = 0.5: Model performs no better than random guessing.
- AUC < 0.5: Model performs worse than random guessing (showing that the model is inverted).



ROC Curve for Evaluation of Classification Models

7. Confusion Matrix

Confusion matrix creates a N X N matrix, where N is the number of classes or categories that are to be predicted. Here we have N = 2, so we get a 2 X 2 matrix. Suppose there is a problem with our practice which is a binary classification. Samples of that classification belong to either Yes or No. So, we build our classifier which will predict the class for the new input sample. After that, we tested our model with 165 samples and we get the following result.

n=165	Predicted No	Predicted Yes
Actual No	50	10
Actual Yes	5	100

There are 4 terms we should keep in mind:

1. **True Positives:** It is the case where we predicted Yes and the real output was also Yes.
2. **True Negatives:** It is the case where we predicted No and the real output was also No.
3. **False Positives:** It is the case where we predicted Yes but it was actually No.
4. **False Negatives:** It is the case where we predicted No but it was actually Yes.